

3 Array

What are Arrays?

An **array** is a **fundamental data structure** present in almost every programming language. Arrays store elements in a **continuous block of memory**.

Key Idea

- Arrays are stored in **contiguous (continuous) memory locations**
- Elements are stored one after another
- Arrays are usually **homogeneous**

Continuous Memory Allocation

Suppose we write in Java:

```
int[] arr = new int[10];
```

If:

- `int` = 4 bytes
- array size = 10

Then total memory required:

$$10 \times 4 = 40 \text{ bytes}$$

These 40 bytes are allocated **continuously in RAM**.

Arrays are Homogeneous

All elements inside an array must be of the **same data type**.

Example:

```
int[] arr = new int[5];  
// allowed 1, 2, 3, 4  
// not allowed 1, "hello", 5.6
```

Why Arrays are Fast?

Arrays are fast because memory is continuous.

This allows direct calculation of any element's address.

$$\text{Address} = \text{BaseAddress} + (\text{Index} \times \text{Size of DataType})$$

Drawbacks of Arrays

Arrays are very fast because they use **continuous memory allocation**. But this same property also creates several limitations.

1. Static Size (Fixed Size)

Traditional arrays have a **fixed size**.

Once the array is created:

- size cannot be changed easily
- memory is already reserved

```
int[] arr = new int[10];
```

- array size = 10
- fixed at creation time

Suppose:

- you created array of size 10
- later you need 100 elements

Problem:

- array cannot grow automatically
- new larger array must be created

Solution

Modern languages solve this using:

- Dynamic Arrays
- ArrayList
- Vector
- Python List

These structures internally resize themselves automatically.

2. Insertion in Beginning or Middle is Inefficient

This is one of the biggest drawbacks.

Suppose array contains:

```
10 20 30 40 50 60 70 80
```

Now insert 35 between 30 and 40 .

What Happens Internally?

To create space:

- 40 shifts right
- 50 shifts right
- 60 shifts right
- 70 shifts right

Then 35 is inserted.

New array:

```
10 20 30 35 40 50 60 70 80
```

Why is This Slow?

Because many elements must move.

If insertion happens at beginning:

- almost all elements shift

This requires iteration through array.

$O(n)$

3. Deletion is Also Inefficient

Suppose we delete 20 .

Before deletion:

```
10 20 30 40 50 60 70 80
```

After deleting 20 :

- all elements shift left

- ```
10 30 40 50 60 70 80
```

## Why Deletion is Slow?

Because:

- elements must move to fill gap
- requires traversal and shifting

$O(n)$

## Why Do These Problems Exist?

Because arrays are:

- Continuous in memory
- Fixed in structure
- This continuity gives:
- fast access
- But causes:
- difficult insertion/deletion

## Abstract Data Type of array

- Access ->  $S, T = O(1)$
- Set ->  $S, T = O(1)$
- Traverse / Search ->  $T = O(n), S = (1)$
- Copy ->  $S, T = O(n)$
- Insert
  - First ->  $T = O(n), S = O(1)$
  - End ->  $T = O(1), S = O(1)$
  - Between ->  $T = O(n), S = O(1)$
- Remove
  - First ->  $T = O(n), S = O(1)$
  - End ->  $T = O(n), S = O(1)$
  - Between ->  $T = O(n), S = O(1)$

## Resizable Arrays (Dynamic Arrays / ArrayList / Vector)

Traditional arrays have a major drawback:

- their size is fixed (static)
- To solve this problem, modern languages provide:
- Dynamic Arrays
- ArrayList
- Vector
- Python List
- These are called **Resizable Arrays**.

```
ArrayList<Integer> arr = new ArrayList<>();
```

Internally:

- a small array is created
- Suppose initial capacity = 2

| Index | Value |
|-------|-------|
| 0     | empty |
| 1     | empty |

```
size = 0
capacity = 2
```

## Amortized Time Complexity

Since resizing does NOT happen every time:  
Average complexity becomes:

$$O(1)$$

This is called: Amortized  $O(1)$

## What is Amortized Complexity?

Amortized complexity means:

- some operations are expensive
  - most operations are cheap
- Average cost over many operations becomes small.

## Drawbacks of Resizable Arrays

### 1. Resizing Cost

Copying elements can be expensive.

### 2. Extra Memory Usage

Unused capacity may exist.

## Creating a List in Python

```
brands = ["Tesla", "Skoda", "Toyota", "Suzuki"]
```

| Operation     | Time Complexity  |
|---------------|------------------|
| Access        | $O(1)$           |
| Append        | $O(1)$ amortized |
| Pop End       | $O(1)$           |
| Insert Middle | $O(n)$           |

| Operation     | Time Complexity |
|---------------|-----------------|
| Remove Middle | $O(n)$          |
| Sort          | $O(n \log n)$   |
| Reverse       | $O(n)$          |

## Array problems

### Problem 1

Given a binary array `nums`, return *the maximum number of consecutive 1's in the array*.

**Input:** `nums = [1,1,0,1,1,1]`

**Output:** 3

**Explanation:** The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

```
from typing import List
class Solution:
 def findMaxConsutiveOnes(self, nums: List[int]) -> int:
 ans = 0
 count = 0
 n = len(nums)
 for i in range(n):
 if nums[i] == 1:
 count = count+1
 if ans < count:
 ans = count
 if nums[i] == 0:
 count = 0
 return ans

s = Solution()
x = s.findMaxConsutiveOnes([1,1,0,1,1,1,0,0,1,1,1,1,0,0,1])
print(x)
```

### Problem 2

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

**Example 1:**

**Input:** `prices = [7,1,5,3,6,4]`

**Output:** 5

**Explanation:** Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit =  $6 - 1 = 5$ .

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

```
from typing import List
class Solution:
 def maxProfit(self, prices: List[int]) -> int:
 minPriceSoFar = prices[0]
 maxProfitSoFar = 0
 n = len(prices)
 for i in range(1, n):
 if minPriceSoFar > prices[i]:
 minPriceSoFar = prices[i]
 if maxProfitSoFar < (prices[i] - minPriceSoFar):
 maxProfitSoFar = prices[i] - minPriceSoFar
 return maxProfitSoFar

s = Solution()
x = s.maxProfit([7, 6, 4, 3, 1])
print(x)
```

### Problem 3

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

You must write an algorithm that runs in  $O(n)$  time.

**Example 1:**

**Input:** `nums = [1,2,3,4]`

**Output:** `[24,12,8,6]`

```
from typing import List
class Solution:
 def productExceptSelf(self, nums: List[int]) -> List[int]:
 pro = 1
 zeroCount = nums.count(0)
 n = len(nums)
 for i in range(n):
 if nums[i] != 0:
 pro = pro * nums[i]

 newLst = []
 for i in range(n):
 if zeroCount >= 2:
 newLst.append(0)
 elif zeroCount == 1:
 if nums[i] == 0:
 newLst.append(pro)
```

```

 else:
 newLst.append(0)
 else:
 newLst.append(pro//nums[i])
 return newLst

```

```

s = Solution()
x = s.productExceptSelf([0,2,0,4])
print(x)

```

## Problem 4

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in  $O(n)$  time and without using the division operation.

**Example 1:**

**Input:** `nums = [1,2,3,4]`

**Output:** `[24,12,8,6]`

## Prefix-Suffix Pattern

$$\prod_{j \neq i} nums[j] = \left( \prod_{j < i} nums[j] \right) \times \left( \prod_{j > i} nums[j] \right)$$

$\prod$  this symbol means multiply everything (product notation)

$\sum$  this means sum

$$\prod_{k=1}^4 k = 1 \times 2 \times 3 \times 4 = 24$$

$j \neq i$  all  $j$  values except  $i$

For index  $i = 2$ :

```
nums = [1, 2, 3, 4]
```

$$\prod_{j < 2} nums[j] = 1 \times 2 \Rightarrow 2$$

$$\prod_{j > 2} nums[j] = 4 \Rightarrow 4$$

multiply both = 8

which is the product of all element except 3.

| Name                  | Used For                    |
|-----------------------|-----------------------------|
| Prefix Sum            | cumulative sums             |
| Prefix Product        | cumulative multiplication   |
| Prefix Array          | generic preprocessing       |
| Suffix Array          | right-side preprocessing    |
| Left-Right Traversal  | two directional computation |
| Preprocessing Pattern | preparing reusable data     |



This pattern is a type of: Dynamic Preprocessing  
Meaning:

- compute reusable information once
- reuse many times

```
from typing import List
class Solution:
 def productExceptSelf(self, nums: List[int]) -> List[int]:
 n = len(nums)
 pre = [1]*n
 post = [1]*n
 ans = [1]*n
 for i in range(1,n):
 pre[i] = pre[i-1]*nums[i-1]
 for i in range(n-2,-1,-1):
 post[i] = post[i+1]*nums[i+1]
 for i in range(n):
 ans[i] = pre[i]*post[i]
 return ans

s = Solution()
x = s.productExceptSelf([1,2,3,4])
print(x)
```

## Problem 5

Given an integer array `nums` , rotate the array to the right by `k` steps, where `k` is non-negative.

**Example 1:**

**Input:** `nums` = [1,2,3,4,5,6,7], `k` = 3

**Output:** [5,6,7,1,2,3,4]

```
from typing import List
class Solution:
 def rotate(self, nums: List[int], k: int) -> List[int]:
 n = len(nums)
 k = k % n
 for i in range(k+1):
 f = nums[0]
 for j in range(n-1):
 nums[j] = nums[j+1]
 nums[n-1] = f
 return nums

s = Solution()
```

```
x = s.rotate([1,2,3,4,5,6,7],3)
print(x)
```

## Sol 2

```
class Solution:
 def rotate(self, nums: List[int], k: int) -> List[int]:
 n = len(nums)
 k = k % n
 nums[:] = reversed(nums[:])
 nums[:k] = reversed(nums[:k])
 nums[k:] = reversed(nums[k:])
 return nums
```

## Problem 6

Given an integer array `nums` , find the `subarray` with the largest sum, and return *its sum*.

**Example 1:**

**Input:** `nums = [-2,1,-3,4,-1,2,1,-5,4]`

**Output:** 6

**Explanation:** The `subarray [4,-1,2,1]` has the largest sum 6.

**Kadane's Algorithm Pattern**

At every index:

You decide:

- continue previous `subarray`
- or discard it and start fresh

The key insight is:

A negative running sum is harmful for future sums.

```
from typing import List
class Solution:
 def maxSubArray(self, nums: List[int]) -> int:
 currentSum = nums[0]
 ans = nums[0]
 n = len(nums)
 for i in range(1, n):
 if currentSum < 0:
 currentSum = 0
 currentSum = currentSum + nums[i]
 if ans < currentSum:
 ans = currentSum
 return ans

s = Solution()
```

```
x = s.maxSubArray([-2,1])
print(x)
```

## Sol 2

```
from typing import List
class Solution:
 def maxSubArray(self, nums: List[int]) -> int:
 maxSum = nums[0]
 ans = nums[0]
 n = len(nums)
 for i in range(1, n):
 maxSum = max(nums[i], nums[i] + maxSum)
 ans = max(maxSum, ans)
 return ans

s = Solution()
x = s.maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4])
print(x)
```

## Problem 7

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

**Note** that the product of an array with a single element is the value of that element.

**Example 1:**

**Input:** `nums = [2,3,-2,4]`

**Output:** 6

```
from typing import List
class Solution:
 def maxProduct(self, nums: List[int]) -> int:
 maxPro = nums[0]
 minPro = nums[0]
 ans = nums[0]
 n = len(nums)
 for i in range(1, n):
 if nums[i] >= 0:
 maxPro = max(nums[i], maxPro * nums[i])
 minPro = min(nums[i], minPro * nums[i])
 else:
 temp = maxPro
 maxPro = max(nums[i], minPro * nums[i])
 minPro = min(nums[i], temp * nums[i])
 ans = max(ans, maxPro)
 return ans
```

```
s = Solution()
x = s.maxProduct([2, -5, -2, -4, 3])
print(x)
```

## Problem 8

Determine if a  $9 \times 9$  Sudoku board is valid. Only the filled cells need to be validated **according to the following rules**:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. Each of the nine  $3 \times 3$  sub-boxes of the grid must contain the digits 1-9 without repetition.

**Note:**

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Input:

board =

```
[["5","3",".",".","7",".",".",".","."],
 ["6",".",".","1","9","5",".",".","."],
 [".","9","8",".",".",".",".","6","."],
 ["8",".",".",".","6",".",".",".","3"],
 ["4",".",".","8",".","3",".",".","1"],
 ["7",".",".",".","2",".",".",".","6"],
 [".","6",".",".",".",".","2","8","."],
 [".",".",".","4","1","9",".",".","5"],
 [".",".",".",".","8",".",".","7","9"]]
```

Output: true

Input:

board =

```
[["8","3",".",".","7",".",".",".","."],
 ["6",".",".","1","9","5",".",".","."],
 [".","9","8",".",".",".",".","6","."],
 ["8",".",".",".","6",".",".",".","3"],
 ["4",".",".","8",".","3",".",".","1"],
 ["7",".",".",".","2",".",".",".","6"],
 [".","6",".",".",".",".","2","8","."],
 [".",".",".","4","1","9",".",".","5"],
 [".",".",".",".","8",".",".","7","9"]]
```

Output: false

```
class Solution:
 def isValidSudoku(self, board: List[List[str]]) -> bool:
```

```

rowSet = [set() for _ in range(9)]
colSet = [set() for _ in range(9)]
gridSet = [set() for _ in range(9)]

for i in range(9):
 for j in range(9):
 if board[i][j] == '.':
 continue
 gridNo = (i//3)*3 + (j//3)

 isPresentInRow = board[i][j] in rowSet[i]
 isPresentInCol = board[i][j] in colSet[j]
 isPresentInGrid = board[i][j] in gridSet[gridNo]

 if isPresentInRow or isPresentInCol or isPresentInGrid:
 return False
 rowSet[i].add(board[i][j])
 colSet[j].add(board[i][j])
 gridSet[gridNo].add(board[i][j])

return True

```

$gridNo = (i//3) * 3 + (j//3)$

Suppose board starts with:

```

5 3 .
6 . .
. 9 8

```

Cell (0,0) = '5'

Check:

- rowSet [0] → empty
- colSet [0] → empty
- gridSet [0] → empty

Add:

```

rowSet[0] = {'5'}
colSet[0] = {'5'}
gridSet[0] = {'5'}

```

Cell (0,1) = '3'

No duplicates.

Add:

```

rowSet[0] = {'5', '3'}
colSet[1] = {'3'}

```

```
gridSet[0] = {'5','3'}
```

## Suppose Later We See Another '5' in Same Row

Check:

```
'5' in rowSet[0]
```

True.

So:

```
return False
```

## Problem 9

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

**Example 1:**

**Input:** `nums = [-4,-1,0,3,10]`

**Output:** `[0,1,9,16,100]`

```
class Solution:
 def sortedSquares(self, nums: List[int]) -> List[int]:
 first = 0
 last = len(nums)-1
 ans = [1]*len(nums)
 i = len(nums)-1
 while i >= 0:
 if (nums[first]**2) > (nums[last]**2):
 ans[i] = nums[first]**2
 first = first+ 1
 else:
 ans[i] = nums[last]**2
 last = last - 1
 i = i - 1
 return ans
```

## Problem 10

An array is **monotonic** if it is either monotone increasing or monotone decreasing.

An array `nums` is monotone increasing if for all  $i \leq j$ , `nums[i] <= nums[j]`. An

array `nums` is monotone decreasing if for all  $i \leq j$ , `nums[i] >= nums[j]`.

Given an integer array `nums`, return `true` if the given array is monotonic, or `false` otherwise.

**Example 1:**

**Input:** nums = [1,2,2,3]

**Output:** true

```
class Solution:
 def isMonotonic(self, nums: List[int]) -> bool:
 isMonoInc = True
 isMonoDec = True
 n = len(nums)
 for i in range(1,n):
 if nums[i] < nums[i-1]:
 isMonoInc = False
 if nums[i] > nums[i-1]:
 isMonoDec = False

 return isMonoInc or isMonoDec
```